



myPOS Checkout API

Accepting online payments

Version 1.4

Document version 1.4.0

myPOS Europe Ltd.



Table of Contents

Table of Contents	2
Version control.....	5
Part 1: General Overview	6
Introduction	6
Scope	7
Structural features of the handbook	7
Terms and descriptions.....	8
Where to start.....	9
Payment Gateway details	9
Implementation basics	9
Use of Trademarks on Merchant's website	13
Currencies accepted at myPOS Checkout API	14
Security and availability	14
Business myPOS Account details.....	16
Integrating myPOS Checkout with online stores.....	16
Adding of an online store.....	16
E-commerce transactions	19
Reserve	19
Details	20
Part 2: Technical integration.....	21
Overview.....	22
HTTP POST.....	22
Data type formats.....	22
Signature and authentications	22
Test environment	24



The process	27
Understanding transmission mechanism	27
Method standard properties.....	28
Response standard properties	28
The calls	28
Purchase with payment card (API call: IPCPurchase)	29
Purchase with iCard Mobile (API call: IPCPurchaseByIcard)	34
Successful payment notification (API call: IPCPurchaseNotify / IPCPurchaseOK).....	37
Cancelation of payment notification (API call: IPCPurchaseCancel)	38
Rollback of previous notification (API call: IPCPurchaseRollback)	39
In-app Purchase with payment card (API call: IPCIAPurchase)	39
Get transaction log for previously executed payment (API call: IPCGetTxnLog)	42
Make a refund for previously executed payment (API call: IPCRefund).....	48
Make a reversal for previously executed payment (API call: IPCReversal).....	49
Store card data for recurring payments (API call: IPCIStoreCard)	50
Update data of stored card for recurring payments (API call: IPCIStoredCardUpdate)	51
Request money from a client's myPOS account (API call: IPCRequestMoney).....	52
Manage registrations for Request money transactions (API call: IPCMandateManagement)	53
Get transaction status for previously executed payment (API call: IPCGetTxnStatus)	54
Make a credit note to a myPOSWallet (API call: IPCSendMoney).....	55
Appendix I – Error messages.....	58
Appendix II – Testing data	59
Test private key	59
myPOS test public certificate	59
Appendix III – Example for successful payment notification	60



Accepting online payments

myPOS Checkout API v.1.4

Result POST data	60
Appendix IV – Card types.....	60
Appendix V – Card verification.....	60



Version control

Date posted	API Version	Document revision	Description
19.08.2015	1.0	1	Version 1.0
13.11.2015	1.0	2	Review of POST requests examples
01.07.2016	1.0	3	Review of POST requests examples
20.12.2016	1.1	0	Version 1.1. Updated calls - IPCPurchase; IPCPurchaseNotify
09.01.2017	1.2	0	Version 1.2
20.02.2017	1.3	0	Version 1.3 Updated calls - IPCPurchase; IPCPurchaseNotify; IPCIAStoreCard; IPCIAStoredCardUpdate
14.03.2017	1.3	1	Version 1.3 / Document revision 1 New calls – IPCRequestMoney, IPCMandateManagement Appendix I and V review Rebranding from myPOS Virtual to myPOS Checkout
19.05.2017	1.3	2	Version 1.3 / Document revision 2 Updated calls / response - IPCPurchaseNotify, IPCIAStoreCard, IPCIAStoredCardUpdate
01.06.2017	1.3	3	Renaming Payment Gateway
11.07.2017	1.3	4	Version 1.3 / Document revision 4 New call – IPCSendMoney
12.12.2017	1.3	5	Version 1.3 / Document revision 5 New call – IPCPurchaseByIcard
30.10.2018	1.4	0	iDEAL Payments

Part 1: General Overview

Introduction

myPOS Checkout API is a customer friendly interface for e-commerce payment solutions for your Online store. The API will let your customers to pay for goods and services quickly and securely using their debit or credit card.

The API will gain access to the entry point of Payment Gateway managed by myPOS Europe Ltd. By using hosted checkout secure pages, the merchant adhere to compliance rules for handling customer data in a secure way: data is stored on security servers so that it is not exposed to compromise.

The API enables you quickly and easily to integrate myPOS Checkout into all your online stores as a method of payment. The integration will give you the opportunity to receive money from your customers instantly, in multi-currencies, to track and manage your payments from all your online stores in a single place and to manage your online stores settings. Therewith the API will give convenience to all your customers to choose their preferred payment method from a single source.

Using the myPOS Checkout API your customers will be temporary redirected to <https://www.mypos.eu/vmypos/>. The Payment Gateway will handle and guide the customer during the payment process, will check the card sensitive data and will process a payment transaction through card schemes (VISA, MasterCard, JCB).

Once the payment is complete, the customer will be returned to the merchant's website. You will receive notification about the payment along with the transaction details.

myPOS Checkout API will provide:

- Secured page and Secured communication channel with the merchant;
- Storing of merchant private data (shopping cart, amount, payment method, transaction details etc.);
- Financial transactions to VISA, MasterCard, JCB (if applicable);
- Operations for the front-end: Purchase transaction, Manage online stores settings;
- 3D secure processing for direct payments with Debit or Credit Cards.



Scope

This handbook is aimed at the operators of e-commerce companies who would like to optimize their payment processes using an innovative and complete payment solution.

The purpose of this document is to specify the myPOS Checkout API Interface and demonstrate how it is used in the most common way, therewith to provide technical details about the system integration.

It is intended to be utilized by:

the merchant / commercial decision makers

Certain sections describe the functions, main requirements and processes of the myPOS Checkout payment system from the stance of e-commerce Business development managers. The document provides guidance on what is a myPOS Checkout API, how it works, getting started, setting up a myPOS Account Number, main benefits of the API and FAQs.

the IT specialists and developers

Other sections of this manual describe the technical background of the myPOS Checkout system. The document provides guidance on technical integration of the API with detailed working examples, use cases and error codes.

Structural features of the handbook



Important notes are indicated by the pictogram on the left.



References to another point in the text or to another document are indicated by the pictogram on the left.

`<?xml .` Expressions or code examples are written in grey background with smaller font size.



Terms and descriptions

API - Application Programming Interface for implementation of the myPOS Checkout system in the merchant e-commerce application.

myPOS Account number - Unique identifier for the Business Account of the merchant.

Online store ID – Unique Identifier for any online store (website) of the merchant which will use the myPOS Checkout API as a payment method.

PAN - Unique payment card number, which identifies the Issuer and the particular cardholder account.

Call – Call by the merchant's e-commerce system to myPOS Checkout API.

Card Validation Code (CVC) or Card Verification Value (CVV) – Three digits encrypted in the magnetic stripe on the back of the card. This is used as a method of verification during credit card processing.

Request - Query or request from the merchant e-commerce system to myPOS Checkout API

Response - Response from myPOS Checkout API to a Call from the e-commerce system.

3-D Secure™ (3DS) - Technical standard developed by Visa, MasterCard and JBC, designed to combat online credit card fraud. Cardholders who have registered for Verify by Visa®, MasterCard SecureCode® or J/Secure® use their password to validate their identity whenever they make a purchase on a participating site.

XML - XML (Extensible Markup Language) is a generally available data format which myPOS Checkout API uses for comfortable exchange with other systems.

Where to start

In order to sign up as an myPOS Checkout API merchant and start using the Payment Gateway, you need to have a Business myPOS Account first.

1. Sign in to your myPOS Account and go to the Online / Online stores.
2. Click on the link Activate.
3. Set up your online stores and generate the keys.



Important! Please have in mind that your stores need to be verified by myPOS Europe Ltd. which will take up to 5 business days. Until your online stores are verified, you can still process transactions but with certain limits.

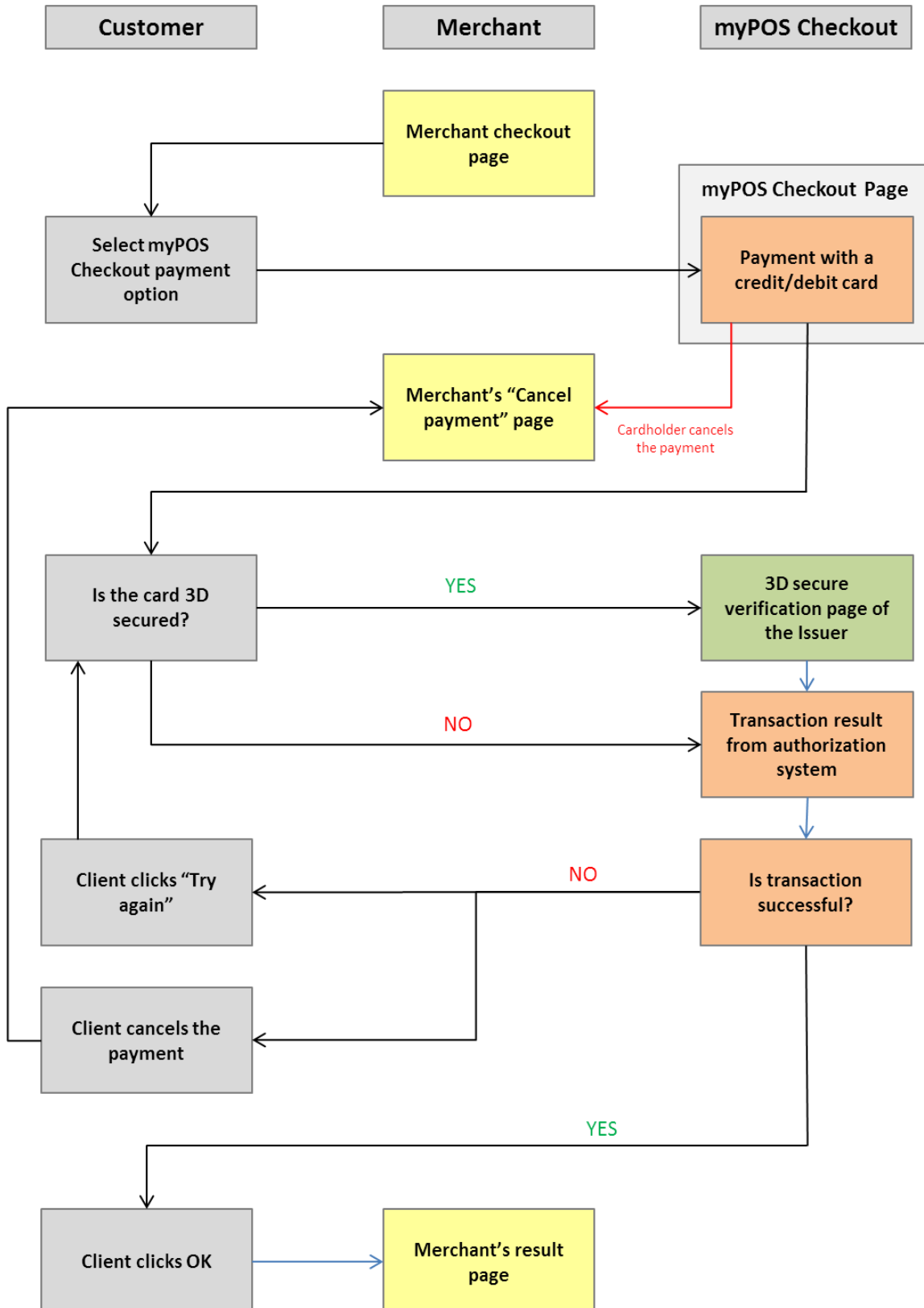
Payment Gateway details

Implementation basics

The API requires the merchants to modify their payment page at the shopping cart so to include myPOS Checkout as a payment option. When the customer chooses myPOS Checkout as a payment method he will actually submit a HTML form to the Payment Gateway secure web servers using HTTPS protocol. The submitted form will contain all needed information about the payment (such as myPOS Account number, online store ID, shopping cart data, currency, amount, etc.)

Once the myPOS Checkout API processes the payment, the system will pass the result to merchant web server and the customer will be redirected to the online store “checkout result” page.

Interaction Diagram





Detailed diagram with all communications and messages between the merchant website and myPOS Checkout API is presented in the [Part 2: Technical integration](#) of this document.

Payment process in steps

1. Internet customer chooses myPOS Checkout as a payment method at the merchant's online store checkout page.
2. Merchant web server initiates payment through the Payment Gateway.
3. Customer web browser is redirected to myPOS Checkout payment link along with the payment details.
4. If customer cancels the payment, he will be redirected to the merchant's "Cancel payment" page.
5. The customer needs to fill in his payment data and to click the button "Confirm".



Most of the customer data will be automatically filled in and the customer will need to complete just few of the fields.

6. Payment gateway system receives the details for the payment – successful or declined.
7. Payment gateway system passes the result to merchant web server and the customer web browser is directed to the myPOS Checkout result page.
8. If the customer clicks on the "Return to ..." button he will be redirected to the merchant's result page.



Accepting online payments

myPOS Checkout API v.1.4

myPOS Checkout payment page



Checkout from
www.example.com

Order No: 2154684351

Item	Item price	Qty	Amount
HP ProBook 6360D sticker	22.50 USD	2	45.00 USD
Delivery	5.00 USD	1	5.00 USD

Total 50.00 USD

This transaction will appear on your statement as MYPOS *EXAMPLE, Germany

Credit or debit card payment details

Country

Cardholder name

Card number

DD / MM

CVC

First name

Last name

ZIP

City

Address

Mobile number (ex.: +44XXXXXXXX)

test@test.test



Paying with myPOS is
safer and easier

Your card details will not be shared with the merchant

CONFIRM

[Cancel and return to www.example.com](http://www.example.com)



MasterCard
SecureCode

Verified by
VISA

If your debit or credit card supports 3-D authentication, you may be taken to the 3-D secure verification page delivered by your bank

support@mypos.eu

Refund transactions

The merchant could initiate a refund transaction for any previously executed payment. This functionality is available in his account pages in menu Transaction/ Transaction details.

In case where the customer has lodged a justified complaint or the merchant is unable to deliver the goods, he could refund any partial amount of the payment back to the customer.



myPOS always books the refund back in the originally used payment method.

The maximum refunded amount is 100 percent of the original amount.

Use of Trademarks on Merchant's website

The Merchant needs to follow the instructions below:

1. General requirements

The myPOS logo and all related with the myPOS Checkout API logos may not be altered, modified, or changed in any way, without the prior written permission of myPOS. These logos may not be used in a manner that refers disparagingly to myPOS or the myPOS Checkout service.

The myPOS Checkout payment option must be shown along with any other payment method on the merchant's website. The service must be presented in a manner which shows service in equality with all other payment methods.

The merchant may not use any of the Card scheme logos (VISA, MasterCard, JCB) separately without the myPOS logo, except in case where the merchant has personal agreement with a specific Card scheme.

The merchant must comply with myPOS any text descriptions of the myPOS Checkout service and/or myPOS before publishing them online.

The merchant may use on his website any logo available on mypos.eu website. If the merchant needs custom size or solution, he must contact myPOS Customer service department.

2. Payment method page

If the merchant has separate payment method page on his website, he needs to describe the myPOS Checkout service, complying with the above General requirements.

3. Checkout page

The myPOS Checkout payment method must be presented on the merchant's checkout page complying with the above General requirements.

4. Redirecting page

If the merchant has a redirecting page between his checkout page and the myPOS Checkout payment page, the merchant needs to have the following attributes on this page:



- <https://www.mypos.eu>
- Payments processed by mypos.eu or myPOS logo

Example:

Please wait, you are now being redirected to <https://www.mypos.eu/vmp/>...

You are about to pay via secure payments processed by myPOS Europe Ltd. using a debit or credit card.

If you are not redirected within 5 seconds, please [click here](#).

The Verified by VISA & MasterCard SecureCode & J/Secure logos could be placed separately in accordance with the requirements of the Card scheme.

Currencies accepted at myPOS Checkout API

myPOS Checkout API is accepting payments and making payouts in all currencies available for myPOS Accounts. Please refer to www.mypos.eu in order to see the current list with all available currencies.

If the merchant needs to accept payments in any other currency, he will need to contact his customer service representative for more information.

Security and availability

Connection between the merchant and the myPOS Checkout API is handled through internet using HTTPS protocol (SSL over HTTP). Requests and responses are digitally signed both. myPOS host is located at Tier IV datacenter in Luxembourg. Public address for myPOS Checkout API is BGP enabled and available through all first level internet providers.

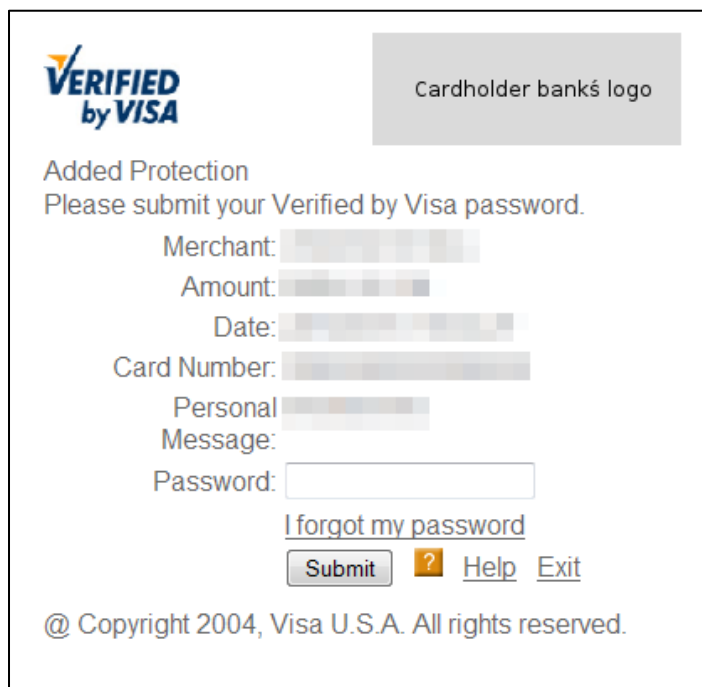
myPOS supplies an emergency support line via e-mail or phone which is 7x24 enabled and reaches certified engineers.

3-D Secure payment

To make online transactions using credit cards safer and more secure, myPOS supports 3-D secure payments.

The service is available for merchant's accounts which support 3-D secure and in case the customer credit card is 3-D secure.

Depending on the Card scheme and the Issuing bank, the customer will see an additional step in the myPOS Checkout payment page. Please take a look at the VISA example below:



Important security requirements for making requests to the myPOS Checkout API

All requests to the API are standard HTTPS requests. The 'User-Agent' HTTP request header is required by myPOS Checkout API.

It is a means of verification of the program on the client host and if the client does not send this string, it cannot be verified nor logged and will result in myPOS Checkout error page with the following text: “The online store has sent myPOS a shopping cart with errors in it. We will contact the Merchant with a request to fix this problem. As this could be a temporary issue, you can go back to try checking out again.” and a link to the merchant’s website.

Sending the 'User-Agent' is one of the principle rules of our network security and is usually a simple setting in client programs. If you are against sending the header for tracking reasons, we inform you that this is used as a loophole by potential attackers.

Security restrictions

Enable/Disable payments for a specific merchant’s online store

By default, the online payment processing for any approved merchant’s online store is disabled. To enable the store the merchant needs to log in his Business Account, to go to the Online / Online stores menu and to click on the button “Enable” beside the particular online store.



The merchant could use the “Enable/ Disable” functionality at any convenient time.

Request URLs

This myPOS feature aims to further increase the security level of the merchant's account, protecting it from unauthorized request attempts.

The merchant must specify at least one URL from which request to the myPOS Checkout API will be made.



All requests from any other URLs will be denied.

The merchant could add new URLs at any time, however all new URLs will be reviewed and approved first.

Signature and public/private key pairs

In every message a signature is supplied.

For signing process, both myPOS Checkout API and the merchant generate public/private key pairs and exchange the public certificate. Key pairs are generated using RSA algorithm. The certificates must be PEM-encoded PKCS7 file. Every of the parties are using the private key to sign the message and the opposite side authenticate the sender with corresponding public certificate.

The myPOS Checkout API provides different myPOS public certificate to everyone online store of the merchant. They are available for download at Online / Online stores / Keys menu.

myPOS Checkout API requires from merchant to upload his public certificate so that his digital signature can be verified from the system. The merchant can upload several public certificates. A key index is assigned to each certificate. For each of the merchant's public certificate there is a certain myPOS public certificate. The merchant can download each myPOS public certificate by clicking on Download in the myPOS public certificate column.



The online store public certificate can be changed at any time from the Online / Online stores / Keys menu.

Business myPOS Account details

Integrating myPOS Checkout with online stores

From menu Online / Online stores the Merchant can activate the service.

Adding of an online store

Once the service is activated the Merchant can start adding the information of the online stores in menu Online / Online Stores / Add new store:

Add new store

[Cancel](#)

Store details

Store name: *

Activity: *

Please, choose...

Billing descriptor: *

Maximum 22 symbols, including MYPOS *. Use Latin letters only.

Website URL: *

Store logo: * ⓘ

Browse

Maximum banner size: 940 x 105 px
File formats: jpg, jpeg, png.
Maximum file size: 64M

Request URLs: ⓘ

Add

Business contact details ⓘ

Customer service URL: *

Customer service email: *

Customer service phone: *

Transaction currencies ⓘ

Currency *	Settlement account *
☐ EUR	
☐ USD	
☐ GBP	
☐ HRK	
☐ CHF	
☐ RON	
☐ JPY	
☐ BGN	

* Required fields

Review and confirm

Setting up the new online store

Once added, the new online store will be visible at the Online / Online Stores / View all stores menu. The new online store will be with status “Disabled”. The merchant needs to:

1. activate the store once he is ready with the test integration with the myPOS Checkout API by clicking on the appropriate button beside the store;
2. read carefully and agree with the General terms and conditions and the Tariff for this store.
3. set key pairs for this particular website;
4. enable the store by clicking on the appropriate button beside the store.



Every online store could be enabled / disabled separately at any time by the merchant.



Accepting online payments

myPOS Checkout API v.1.4

myPOS™ Helps my business grow.

13:04 EN Current client: [Logout](#) / [Switch](#)

Accounts Devices Cards Transfers **Online** Merchant services Profile

Pay Buttons and Links

Online Stores

[View all stores](#)

[Add new store](#)

[Transactions](#)

[Shopping carts](#)

[Integration Support](#)

My online stores

Store ID	Billing descriptor	Store name	URL	Used Currencies	Tariff	Status	Actions
	MYPOS *STORE NAME 1	STORE NAME 1	www.website.com	EUR - EUR / EUR Account USD - USD / / USD account	View	Disabled	Enable Keys Edit
	MYPOS *STORE NAME 2	STORE NAME 2	www.website2.com	EUR - EUR / / EUR Account	View	Enabled	Disable Keys Edit

If you need help setting up your integration, please visit [Integration Support](#).

myPOS package and platform are promoted and operated by myPOS Europe Ltd, London, UK.
The payment services and accounts for e-money for EEA countries are provided by designated licensed banks or e-money institutions.

[General terms and conditions](#) [Tariff](#) [Currency Exchange Rates](#) [myPOS SEPA Direct Debit Schedule](#) [Return Policy](#) [Privacy Policy](#) [Acceptance Policy](#) [Return Material Authorization](#) [List of Third Parties](#)

Contacts [sales@mypos.eu](#) [support@mypos.eu](#)



myPOS™ Helps my business grow.

13:37 EN Current client: [Logout](#) / [Switch](#)

Accounts Devices Cards Transfers **Online** Merchant services Profile

Pay Buttons and Links

Online Stores

[View all stores](#)

[Add new store](#)

[Transactions](#)

[Shopping carts](#)

[Integration Support](#)

Keys for store "STORE NAME 1"

[Back](#)

The myPOS Checkout API requires the use of key pairs' cryptography to ensure that the identity of the sender is guaranteed. It works through the use of public key and private certificate.

For signing process, both myPOS and the Merchant generate public and private key pairs and exchange the public certificate. Every of the parties are using the private key to sign the message and the opposite side authenticate the sender with corresponding public certificate. Key pairs are generated using RSA algorithm. The certificates must be PEM-encoded PKCS7 file.

Generate and upload your STORE NAME 1 myPOS Public Certificate

The merchant can upload several public certificates. A key index is assigned to each certificate. The key index of the certificate used to sign the request should be supplied as a parameter in every transmission. For each of the merchant's public certificate there is a certain myPOS public certificate. The merchant can download each myPOS public certificate by clicking on Download in the myPOS public certificate column.

Keys						Add new certificate
Key index	Added on	Custom name	Certificate name	Expiration date	Actions	myPOS public certificate
1	28.09.2016 14:26	Key 1	/O=iPay	26.08.2026	Download Delete	Download
2	16.03.2017 09:33	Key index 2	/C=BG/O=iPay	14.02.2027	Download Delete	Download
3	17.03.2017 09:54	Key index 3	/C=BG/O=iPay	15.02.2027	Download Delete	Download

If you prefer, you could [Generate new key pair](#) for your online store here, save them and upload your public key above.

If you use myPOS Online with a shopping cart you need to:

1. Generate new key pair for your online store.
2. Save them to a text file or similar.
3. Copy your private key to the Online platform » myPOS Online payment method settings.
4. Upload your public certificate above.
5. Copy the relevant myPOS public certificate to the Online platform » myPOS Online payment method settings.

[Generate new keys](#)

If you need help setting up your integration, please visit [Integration Support](#).

Editing the online store data

The merchant can change the main information of every online store using the “Edit” functionality at the Online / Online Stores.

E-commerce transactions

Overview

All e-commerce transactions will be visible in the merchant’s myPOS Account under menu Online / Online Stores / Transactions. The merchant could filter and see only the transactions which come from myPOS Checkout or from a specific online store.

Details of e-commerce transactions

The merchant will see the following type of details for the e-commerce transactions:

- Payment details – such as payment origin and used payment method;
- Transaction details – transaction reference, exchange rate, amount, transaction fee;
- Reserve – reserve amount from the specific transaction and reimbursement date;
- Cart details – all data from the shopping cart shown to the customer during the checkout, i.e. item name, price, quantity, order total and note;
- Refund details / functionality – refund functionality with which the merchant could initiate partial or full amount refund transaction to a previously executed payment. If there is at least one refund transaction for the initial payment the merchant will see it here together with refund transaction reference and refund amount and fee.

Making a refund

In case where the customer has lodged a justified complaint or the Merchant is unable to deliver the goods, he could refund any amount of the payment back to the customer. The merchant can refund up to 100 percent of the original amount.

When the merchant initiate a refund, a reference will be added to the original transaction details along with new row for the refund transaction.



Once the merchant issues a refund, it cannot be cancelled.

If there are insufficient funds in the merchant’s myPOS Account, the refund transaction will be denied.



If the myPOS Account balance becomes negative as a result of deducted refunds or chargebacks, myPOS Europe Ltd. will collect funds from the merchant in accordance with the Legal agreements.

Reserve

Overview

The reserve is a percentage of money that must remain in the merchant’s myPOS Account for a specific period of time to cover any payment reversals that may be received like chargebacks, claims, and disputes.

The reserve rate and period is specified in the merchant's tariff and depends on the risk assessment of the particular online store. They will be reviewed regularly and adjusted when necessary.



If the merchant Business myPOS Account is blocked by Issuer or closed for any reason, myPOS may hold the reserve for up to 180 days after the date of the last transaction with myPOS Checkout.

Details

The merchant can track the accumulated currency reserve amount and how exactly they are calculated at any time. He can filter and see all transactions which were calculated in it and when the particular amount will be reimbursed.



Part 2: Technical integration



Overview

HTTP POST

Data transfer between Merchant and myPOS Checkout is made by HTTP POST. All the parameters for the requests are in the body in [parameter=value] form. Separator between tokens is [&]. The body is URL Encoded. Character encoding is UTF-8.

Example:

```
POST /somescript.php HTTP/1.1
Host: www.somesite.com
User-Agent: Mozilla/4.0
Content-Length: 39
Content-Type: application/x-www-form-urlencoded
userid=joe&password=guessme&user_type=1
```

Data type formats

Data Type in document	Description	Example
int	Integer	1
String	String	This is a string
Date	ISO 8601 date string YYYY-MM-DD	2012-03-31
DateTime	ISO 8601 datetime string YYYY-MM-DD HH:mm:ss	2012-03-31 23:59:59
A(n)	Alpha string. [n] characters required	Alpha string
AN(n)	Alphanumeric string. [n] characters required	Alphanumeric string
N(n)	Numeric string. [n] characters required. Number is left-padded with zeroes.	000123
double	Numeric string with decimal point. Only point is used (no commas or other characters for decimal point)	34.56
BASE64	String used to pass binary data. The binary data should be converted to base64 standard.	YW55IGNhcm5hbCBwbGVhc3VyZQ==
XML	Simple in place XML array.	<body> <param>1</param> <value>2</value> </body>

Signature and authentications

Signatures are calculated using the following mechanism. All data in POST request without the Signature property are concatenated with dash and Base64 encoded. The string is signed with the private key using the SHA-256 algorithm. Then the signature needs to be Base64 encoded. The signature property is added to the POST request.

The opposite side should concatenate all data in the POST request without the Signature property, Base64-encode the string and then verify the obtained string with the sent signature property and the public key extracted from the myPOS public certificate.



The Merchant should always verify the signature when receiving a call from myPOS Checkout API!

Example for PHP 5.x.x

```
<?php

// The POST data array
$postData = array('IPMethod'=>'IPCPurchase', .....);

// This is an example of RSA private key
$privKey = '-----BEGIN RSA PRIVATE KEY-----
MIICXAIBAAKBgQCf0TdcTuphb7X+Zwekt1XKEWZDczSGecfo6vQfqvraf5VPzcnJ
2Mc5J72HBm0u98EJHan+nle2WOZMVGlTa/2k1FRWwbt7iQ5dzDh5PEeZASg2UWe
hoR8L8MpNBqH6h7ZITvTfRS4LSbVlEft7Pzhm5YJKfM+CdzDM+L9WVEGwIDAQAB
AoGAYfKxwUtebq8ulVrD3nnWhF+hk1k6KejdUq0dLYN29w8WjbCMKb9IaokmqWiQ
5iZGERYxh7G4BDP8AW/+M9HXM4oqm5SEkaxhbTlgks+E1s9dTpdFQvL76TvodqSy
l2E2BghVgLLgkdhRn9buaFzYta95JKfgyKGonNxsQA39PwECQQDKbG0Kp6KEkNgB
srCq3Cx2od5OfiPDG8g3RYZKx/O9dMy5CM160DwusVJpuywbpRhCWr3gkz0QgRMd
IRVwyXNbAkeAyh3sipmcgN7SD8xBG/MtBYpQWP1vxhSVYPfJzuPU3gS5MRJzQHBz
sVCLhTBY7hHSoqilqWYasi81JzBEwEuQQJBaKw9qGcZjyMH8JU5TDSGllr3jybx
FFMPj8TgJs346AB8ozqLL/ThvWPpxHttJbH8QAdNuyWdg6dlfVAa95h7Y+MCQEZg
jRDI1Bz7eWGO2c0Fq9OTz3IVLWpnmGwfW+HyaxizxHfV+FOj1GUVir9hylV7V0DU
Qjlajyv/oeDWhFQ9wQECQCydhJ6NaNQOCZh+6QTrH3TC5MeBA1Yeipoe7+BhsLNr
cFG8s9sTxRnltcZl1dXaBSemvpNvBizn0Kzi8G3ZAgc=
-----END RSA PRIVATE KEY-----';

// You need to concatenate all values from $postData and to Base64-encode the result
$concData = base64_encode(implode('-', $postData));
$privKeyObj = openssl_get_privatekey($privKey);

// Signed data in binary
openssl_sign($concData, $signature, $privKeyObj, OPENSSL_ALGO_SHA256);

// Base64 encoding of the signature
$signature = base64_encode($signature);

// Now you need to add the signature to the POST request
$postData['Signature'] = $signature;
openssl_free_key($privKeyObj);

?>
```

Signature verification example for PHP 5.x.x

```
<?php

// Save POST request data in var $data
$data = $_POST;

// myPOS certificate
$cert = '-----BEGIN CERTIFICATE-----
```



```
MIIBkDCB+qADAgECAGAwDQYJKoZIhvcNAQEFBQAwDzENMA5GA1UEChMEaVBheTAe
Fw0xMzAzMTMxMTI1MTFaFw0yMzAzMTExMTI1MTFaMA8xDTALBgNVBAoTBGIlQYXkw
gZ8wDQYJKoZIhvcNAQEBBQADgY0AMIGJAoGBAML+VTmiY4yChoOTMZXAG/mk+x
f/9mjwHxWxtBJbNncNK0OLI0VXYKW2GgVklGHHQjvew1hTFkEGjnCJ7f5CDnbgx
evtyASDGst92a6xcAedEadP0nFXhUz+cYYIlgcfDcX3ZWeNEF5kscqy52kpD2O7
nFNCV+85vS4duJBNAgMBAAEwDQYJKoZIhvcNAQEFBQADgYEAfSfqJHH9Vp9Y4osJ
sLg1Um5LOoTgn6u4JepHMFoSiwYEOn/N3D3JlgqAzjdVJ+1rZV95VAf/+TKzWzvP
V8L01LJ8aRFkUaPGenVsGvBT2mtsbu34QUOIPgzCi3huidwk0ylMX7zo8uxu1cXv
/bg5jBGe5SjvJP8Tq257QcAGgkA=
-----END CERTIFICATE-----';

// Save signature
$signature = $data['Signature'];
// Remove signature from POST data array
unset($data['Signature']);
// Concatenate all values
$concData = base64_encode(implode('-', $data));

// Extract public key from certificate
$pubKeyId = openssl_get_publickey($cert);

// Verify signature
$res = openssl_verify($concData, base64_decode($signature), $pubKeyId, OPENSSL_ALGO_SHA256);
//Free key resource
openssl_free_key ($privKeyObj);

if($res==1){
    //success
}else{
    //not success
}

?>
```

Test environment

Overview

The myPOS Checkout API test environment is a self-contained, virtual testing environment. Once you are ready with the API integration, you need to use it in order to make the appropriate tests of all your myPOS Checkout API calls and communication with the system. The test environment will let you test the API integration without affecting any real myPOS users or their real myPOS Wallets.

The myPOS Checkout API test environment mimics myPOS's production servers, so you can format and make all your Calls to the system as you will do in Production environment. The only difference between the Calls on both environments is that you need to use different user credentials and endpoints.

Before you start using myPOS Checkout into production, you should test the application with all case scenarios in myPOS Checkout API test Environment.



Testing data

Test environment host: <https://www.mypos.eu/vmp/checkout-test>

SID: 000000000000010

Wallet number: 61938166610

POST parameters example

```
IPCmethod=IPCPurchase&
IPCVersion=1.3&
IPLanguage=EN&
SID=000000000000010&
walletnumber=61938166610&
Amount=23.45&
Currency=EUR&
OrderID=20120331999999&
URL_OK=http://site.ext/paymentOK&
URL_Cancel=http://site.ext/paymentNOK&
URL_Notify=https://site.ext/paymentNotify&
CardTokenRequest=0&
KeyIndex=1&
PaymentParametersRequired=1&
customeremail=name@website.com&
customerfirstnames=John Santamaria&
customerfamilyname=Smith&
customerphone=+23568956958&
customercountry=DEU&
customercity=Hamburg&
customerzipcode=20095&
customeraddress=Kleine Bahnstr. 41&
Note=&
CartItems=2&
Article_1=HP ProBook 6360b sticker&
Quantity_1=2&
Price_1=10&
Currency_1=EUR&
Amount_1=20&
Article_2=Delivery&
Quantity_2=1&
Price_2=3.45&
Currency_2=EUR&
Amount_2=5&
Signature=TuQ6nQJxWJ2T+sM6uzBqYYtOWdw+0ecHaRzujTJChWds/1HWK+kCcfKrAW9sN8xzsRBSZ2zH1uPMMIMgB3XqqHN
bq06YhpD3XY/Ltp+ooc8xoq1jdajnUexC5JuDzTslCMKKFmW5vl0HsEkPltyxir0Z5AWpgOZkjqCPEr817/o
```



Please, have in mind that you could test with every debit or credit card number. In test environment all card transactions will be processed as successful payment. Your card will not be charged!

Test scenarios

In order to start testing the myPOS Checkout API payments, you need to:

1. Implement the myPOS Checkout API.



2. Send a POST request with all the data to the test host address.
3. Test the following scenarios:
 - Error in the POST request – missing or invalid data;
 - Send wrong signature to myPOS Checkout API;
 - Check myPOS Checkout API message signature for authenticity;
 - The Customer terminates the process on the myPOS Checkout payment page;
 - Customer pays successfully;
 - Customer pays successfully, however the merchant's website does not return HTTP OK to myPOS Checkout API.

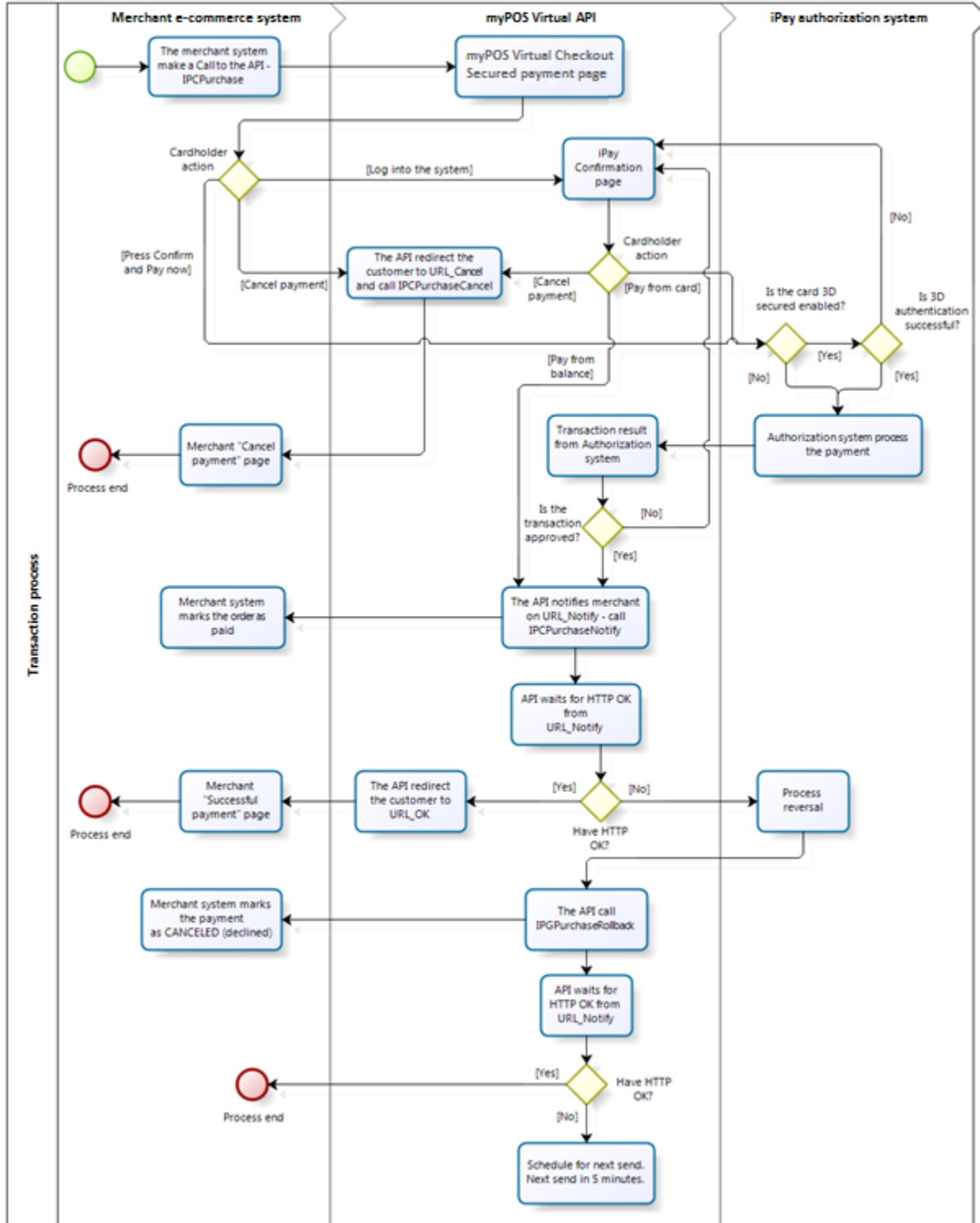
Production environment

Once you have everything implemented and tested, you need to set your online store keys to production environment and change your test data.

Production environment host: <https://www.mypos.eu/vmp/checkout>

The process

Understanding transmission mechanism



Method standard properties

In every request there are several parameters that are always supplied. Below they are called 'standard properties'. Once defined below, they won't be described in every single command listed in the specification, they should be considered as existing to every command.

Property	Typical value	Type	Description
IPCmethod	IPCPurchase	String	Name of the method requested for execution from IPC.
Signature	Byte[]	BASE64	SHA-256 HASH for all properties in the command. Signature is ALWAYS THE LAST PARAMETER IN THE POST , as it is not used to calculate the hash.
KeyIndex	1	Int	Identifier of the private key used for signature (if more than 1).
IPCVersion	1.3	string	Version of protocol used for transition.
IPCLanguage	EN	A(2)	ISO 2-character code for the desired language on the payment page. If myPOS Checkout API cannot fulfill the requested language, it will set the English language as defaults. Currently supporting EN, FR, IT, DE, SV, PT, NL, EL, BG, ES.
WalletNumber	61938166610	N(11)	myPOS Account Number
SID	0000000000000010	AN(15)	Site ID. Identifier of the website accepting payment.

Response standard properties

Upon HTTP request, the party should respond with header HTTP 200 OK with the following body content: "OK". Every other response should be treated as communication error, call error, server error or system malfunctions.

Property	Typical value	Type	Description
IPCmethod	IPCPurchase	String	Name of the method requested for execution from IPC.
Status	0	N(3)	Code upon command completion. 0 is success otherwise error.
StatusMsg	Success	C(max)	Description for <status> code.
Signature	Byte[]	BASE64	SHA-256 HASH for all properties in the command. Signature is ALWAYS THE LAST PARAMETER IN THE POST , as it is not used to calculate the hash.

Once defined these parameters should be considered as existing to every response listed in the specification.

The calls

API function call	Description
MERCHANT TO MYPOS CHECKOUT	
IPCPurchase	This is the standard method for checkout at a shop. The method can be used for card storage also. All sensitive data will be collected on myPOS Checkout Payment page. Customer interaction needed.



IPCPurchaseByIcard	This method enables a myPOS merchant to accept payments on a website using iCard mobile application.
IPCIAPurchase	This is a standard method for checkout when the purchase will be embedded within the external app. All sensitive data will be collected from the external application.
IPCIASoreCard	This is a method for storing a card data. All sensitive data will be collected from the external application.
IPCIASoreCardUpdate	This is a method used to update a data of already stored card. All sensitive data will be collected from the external application.
IPCRefund	Credit to a customer, e.g. return money. No customer interaction needed.
IPCReversal	This method cancels a previously executed payment (void). No customer interaction needed.
IPCRequestMoney	Make a direct withdrawal transaction (Request money) from a specific myPOS account (wallet). No customer interaction needed.
IPCMandateManagement	Make a new registration of a mandate reference for Request money transactions from a myPOS account or cancel a previously registered mandate reference. No customer interaction needed.
IPCGetTxnLog	Returns detailed information about a previously executed payment. No customer interaction needed.
IPCGetTxnStatus	Returns the status and the parameters of a previously executed payment. No customer interaction needed.
IPCSendMoney	Credit note to a myPOS Wallet. No customer interaction needed.
MYPOS CHECKOUT TO MERCHANT	
IPCPurchaseNotify	myPOS Checkout will respond with this method on successful payment. The call will be made on the previously supplied URL_Notify.
IPCPurchaseOK	myPOS Checkout will redirect with this method on successful payment. The call will be made on the previously supplied URL_OK.
IPCPurchaseCancel	myPOS Checkout will redirect with this method when the customer chooses to cancel the payment. The call will be made on the previously supplied URL_Cancel.
IPCPurchaseRollback	myPOS Checkout will notify that a reversal is passed for a previous successful authorization. The merchant should mark the order as not paid (in case, the merchant has received IPCPurchaseNotify method). This is used when myPOS Checkout API does not receive an HTTP OK from the merchant as a response for the IPCPurchaseNotify method.

All commands described below do not include the standard properties discussed in the previous topic. However, the standard properties are mandatory for all commands.

Purchase with payment card (API call: IPCPurchase)

Purpose

This method initiates the beginning of the payment process for a customer. The customer is placed on a page that requests entering payment card details.

myPOS Checkout will check for:

- Valid myPOS Account number



- Valid Online Store ID corresponding with this myPOS Account number
- Valid status of the Online store (enabled)
- Valid currency and total amount
- Valid signature

Method properties

Property	Typical value	Type	Required	Description
Amount	23.45	Double	YES	The amount of the payment requested.
Currency	EUR	A(3)	YES	ISO 3-character currency code. The currency for the payment should be registered and approved.
OrderID	20120331999999	String	YES	Placeholder for the merchant. Used to put some data that will help the merchant to recognize for which order is the payment. Up to 255 characters.
URL_OK	http://site.ext/paymentOK	String	YES	The page where the cardholder should be redirected on successful payment.
URL_Cancel	http://site.ext/paymentNOK	String	YES	The page where the cardholder should be redirected when <Cancel> is pressed on the payment page.
URL_Notify*	https://site.ext/paymentNotify	String	YES	Address supplied by the partner, where the IPCPurchaseNotify API call will send the parameters for the successful payment.
CardTokenRequest	0	N(1)	YES	0 – Do not request a payment card token 1 – Store new card and request a token 2 – Pay with a card and request a token Token will be available in IPCPurchaseNotify callback.



Accepting online payments

myPOS Checkout API v.1.4

PaymentParametersRequired	1	N(1)	YES	1- Full request (All parameters described as conditional are required in the request. The client will not be able to edit them on the payment page.) 2 – Simplified request (All parameters described as conditional are not required in the request. The client will be forced to fill them in on the payment page.) 3 – Simplified request & Simplified payment page (All parameters described as conditional are not required in the request. On the payment page the client will see and fill in only the payment card details.)
CustomerEmail	name@website.com	String	Conditional	<i>Conditional.</i> The parameter is required when PaymentParametersRequired = 1 This is a customer's email.
CustomerPhone	+23568956958	String	NO	This is a customer's phone.
CustomerFirstNames	John Santamaria	String	Conditional	<i>Conditional.</i> The parameter is required when PaymentParametersRequired = 1 All customer's names without the surname.
CustomerFamilyName	Smith	String	Conditional	<i>Conditional.</i> The parameter is required when PaymentParametersRequired = 1 The customer's surname.
CustomerCountry	DEU	String	NO	ISO country code
CustomerCity	Hamburg	String	NO	
CustomerZIPCode	20095	String	NO	
CustomerAddress	Kleine Bahnstr. 41	String	NO	Customer's address.
AccountSettlement	1111111119	N(11)	NO	Account for payment settlement
Note		String	NO	Text associated with the purchase.
CartItems	2	Int	Conditional	The number of rows (items) in the logical record Cart. If there will be some additional fees/taxes for the cardholder, they need to be added as new items.
Cart	Logical Holder	Logical Record	Conditional	Array provided by the Merchant. The array describes the content of the shopping cart. The content will be displayed on the myPOS Checkout API payment page.



*** The URL_Notify URL should be SSL-enabled address only (i.e. it must start with "https://"). Unsecure URLs will be treated as wrong.**

Same OrderID is used for the request of a partner, except in case where the order has been marked as paid. Then OrderID is a unique identifier and myPOS Checkout API will reject duplicated transmission.

Cart Logical Record

Cart logical record consists of standard POST parameters with the form name=value. For every consequent item an index is added that shows the logical record number for the item (ex. Article_1). Indexes are from 1 to <CartItems>.

Property	Typical value	Type	Description
Article	HP ProBook 6360b sticker	String	Name of an article in the shopping cart.
Quantity	2	Int	How many pieces of an article.
Price	2.34	Double	Price of a single unit.
Amount	4.68	Double	Quantity*Price for the article.
Currency	EUR	A(3)	Should be the same currency as in the purchase amount.

Example

New lines and tabulators are included for better reading and do not exist in the POST request.

```
IPCmethod=IPCPurchase&
IPCVersion=1.3&
IPLanguage=EN&
SID=0000000000000010&
walletnumber=61938166610&
Amount=23.45&
Currency=EUR&
OrderID=20120331999999&
URL_OK=http://site.ext/paymentOK&
URL_Cancel=http://site.ext/paymentNOK&
URL_Notify=https://site.ext/paymentNotify&
CardTokenRequest=0&
KeyIndex=1&
PaymentParametersRequired=1&
customeremail=name@website.com&
customerfirstnames=John Santamaria&
customerfamilyname=Smith&
customerphone=+23568956958&
customercountry=DEU&
customercity=Hamburg&
```




Accepting online payments

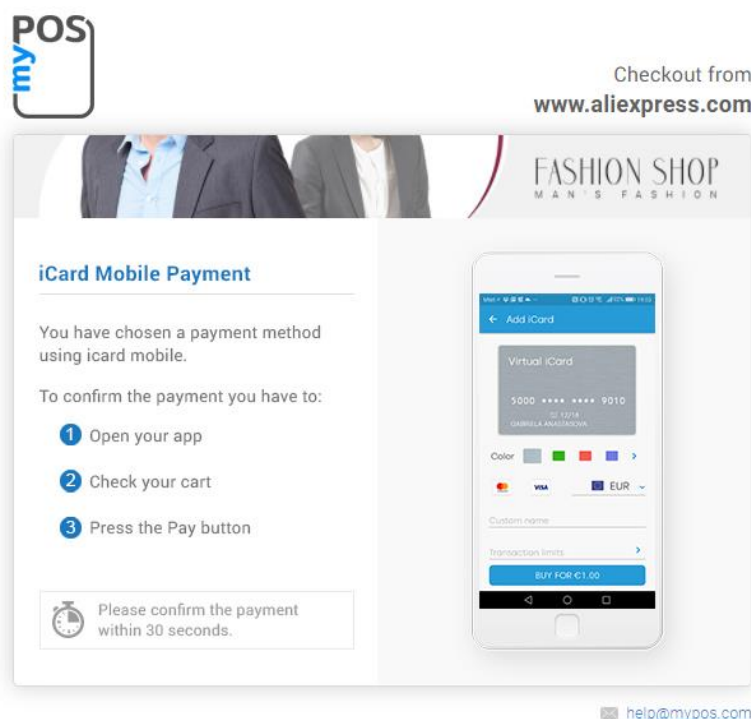
myPOS Checkout API v.1.4

```
customerzipcode=20095&
customeraddress=Kleine Bahnstr. 41&
Note=&
CartItems=2&
Article_1=HP ProBook 6360b sticker&
Quantity_1=2&
Price_1=10&
Currency_1=EUR&
Amount_1=20&
Article_2=Delivery&
Quantity_2=1&
Price_2=3.45&
Currency_2=EUR&
Amount_2=5&
Signature=TuQ6nQJxWJ2T+sM6uzBqYYtOWdw+0ecHaRzujTJChWds/1HWK+kCcfKrAW9sN8xzsRBSZ2zH1uPMMIMgB3XqqHN
bq06YhpD3XY/Ltp+ooc8xoq1jdajnUexC5JuDzTsICMKKFmW5vl0HsEkPltyxir0Z5AWpgOZkjQCPEr817/o=
```

Purchase with iCard Mobile (API call: IPCPurchaseByIcard)

Purpose

The method IPCPurchaseByIcard enables the iCard Mobile application checkout on a myPOS merchant's website. This means that an iCard Mobile user can pay to a myPOS merchant who has integrated this method on the website. Upon selection of iCard Mobile as a payment method on the merchant's website, customer will see an info screen and will instantly receive a notification for payment on the mobile device where iCard Mobile is installed and running. Customer must confirm the payment on their mobile device. No further interaction on merchant's website is needed. After the payment is completed/rejected, the customer will be informed of the result on the merchant's website and in the mobile app.



Payment screen example - iCard Mobile

myPOS Checkout will check for:

- Valid myPOS Account number – where money will be settled
- Valid Online Store ID corresponding with this myPOS Account number
- Valid status of the Online store (enabled)
- Valid currency and total amount
- Valid signature
- Valid customer e-mail or phone number enrolled with iCard mobile



Method properties

Property	Typical value	Type	Required	Description
Amount	23.45	Double	YES	The amount of the payment requested.
Currency	EUR	A(3)	YES	ISO 3-character currency code. The currency for the payment should be registered and approved.
OrderID	20120331999999	String	YES	Placeholder for the merchant. Used to put some data that will help the merchant to recognize for which order is the payment. Up to 255 characters.
URL_OK	http://site.ext/paymentOK	String	YES	The page where the cardholder should be redirected on successful payment.
URL_Cancel	http://site.ext/paymentNOK	String	YES	The page where the cardholder should be redirected when <Cancel> is pressed on the payment page.
URL_Notify*	https://site.ext/paymentNotify	String	YES	Address supplied by the partner, where the IPCPurchaseNotify API call will send the parameters for the successful payment.
CustomerEmail	name@website.com	String	Conditional	<i>Conditional.</i> The parameter is required when CustomerPhone is not provided This is a customer's email.
CustomerPhone	+23568956958	String	<i>Conditional</i>	The parameter is required when CustomerEmail is not provided Must be in International Phone Numbers Format (E.123): (+)(country code)(client number)
Note		String	NO	Text associated with the purchase.
CartItems	2	Int	YES	The number of rows (items) in the logical record Cart. If there will be some additional fees/taxes for the cardholder, they need to be added as new items.
Cart	Logical Holder	Logical Record	YES	Array provided by the Merchant. The array describes the content of the shopping cart. The content will be displayed on the myPOS Checkout API payment page.

* **The URL_Notify URL should be SSL-enabled address only (i.e. it must start with "https://"). Unsecure URLs will be treated as wrong and request will be unsuccessful.**



Same OrderID is used for the request of a partner, except in case where the order has been marked as paid. Then OrderID is a unique identifier and myPOS Checkout API will reject duplicated transmission.

Cart Logical Record

Cart logical record consists of standard POST parameters with the form name=value. For every consequent item an index is added that shows the logical record number for the item (ex. Article_1). Indexes are from 1 to <CartItems>.

Property	Typical value	Type	Description
Article	HP ProBook 6360b sticker	String	Name of an article in the shopping cart.
Quantity	2	Int	How many pieces of an article.
Price	2.34	Double	Price of a single unit.
Amount	4.68	Double	Quantity*Price for the article.
Currency	EUR	A(3)	Should be the same currency as in the purchase amount.

Example

New lines and tabulators are included for better reading and do not exist in the POST request.

```
IPCMethod= IPCPurchaseByIcardMobile &
IPCVersion=1.3&
IPLanguage=EN&
SID=0000000000000010&
walletnumber=61938166610&
Amount=23.45&
Currency=EUR&
OrderID=20120331999999&
URL_OK=http://site.ext/paymentOK&
URL_Cancel=http://site.ext/paymentNOK&
URL_Notify=https://site.ext/paymentNotify&
KeyIndex=1&
customeremail=name@website.com&
customerfirstnames=John Santamaria&
customerfamilyname=Smith&
customerphone=+23568956958&
customercountry=DEU&
customercity=Hamburg&
customerzipcode=20095&
customeraddress=Kleine Bahnstr. 41&
Note=&
CartItems=2&
Article_1=HP ProBook 6360b sticker&
```



```
Quantity_1=2&
Price_1=10&
Currency_1=EUR&
Amount_1=20&
Article_2=Delivery&
Quantity_2=1&
Price_2=3.45&
Currency_2=EUR&
Amount_2=5&
Signature=TuQ6nQJxWJ2T+sM6uzBqYYtOWdw+0ecHaRzujTJChWds/1HWK+kCcfKrAW9sN8xzsRBSZ2zH1uPMMIMgB3XqqHN
bq06YhpD3XY/Ltp+ooc8xoq1jdajnUexC5JuDzTslCMKKFmW5vl0HsEkPltyxir0Z5AWpgOZkjQCPeR817/o=
```

Successful payment notification (API call: IPCPurchaseNotify / IPCPurchaseOK)

Purpose

This method is used by myPOS Checkout to notify the merchant for a successful payment and to pass all needed parameters for the payment on URL_Notify. After successful response for this method myPOS Checkout will redirect the customer browser to URL_OK and will pass same parameters with IPCPurchaseOK method.

Method properties

Property	Typical value	Type	Description
Amount	23.45	Double	<i>Conditional.</i> Echo from IPCPurchase.
Currency	EUR	A(3)	Echo from IPCPurchase.
OrderID	201203319999999	string	Echo from IPCPurchase.
IPC_Trnref	12345678923	String	Used to uniquely identify a transaction in IPC. Used as a parameter for subsequent refund of reversal if needed.
RequestDateTime	2012-03-31 23:59:59	DateTime	Date/time of the request
RequestSTAN	123456	N(6)	Consequent number from 1 to 999999. Used for request unique match.
CardToken	Byte[]	BASE64	<i>Conditional.</i> The CardToken is returned when the card is stored (depending on the chosen option in CardTokenRequest).
PAN	4885	N(4)	<i>Conditional.</i> Last four digits of the account number (PAN). The PAN is returned when the card is stored (depending on the chosen option in CardTokenRequest).
ExpDate	1703	N(4)	<i>Conditional.</i> The ExpDate is returned when the card is stored (depending on the chosen option in CardTokenRequest).



CardType	1	N(1)	<i>Conditional.</i> The CardType is returned when the card is stored (depending on the chosen option in CardTokenRequest).
CustomerEmail	name@website.com	String	<i>Conditional.</i> When PaymentParametersRequired = 1; 2
CustomerPhone	+23568956958	String	<i>Conditional.</i> When PaymentParametersRequired = 1; 2
CustomerFirstNames	John Santamaria	String	<i>Conditional.</i> When PaymentParametersRequired = 1; 2
CustomerFamilyName	Smith	String	<i>Conditional.</i> When PaymentParametersRequired = 1; 2
Signature	Byte[]	BASE64	SHA-256 HASH for all properties in the command. Signature is ALWAYS THE LAST PARAMETER IN THE POST , as it is not used to calculate the hash.

The IPCGetTxnStatus call will return the parameters of the response.

Example

```
HTTP/1.1 200 OK
Date: Mon, 11 Mar 2013 08:04:50 GMT
Server: Apache/2.2.22 (Win32) PHP/5.2.17
X-Powered-By: PHP/5.2.17
Set-Cookie: PHPSESSID=a048e995f78f9e38b192f528f67d3cc3; path=/
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate, post-check=0, pre-check=0
Pragma: no-cache
Content-Length: 2
Connection: close
Content-Type: text/html; charset=utf-8
```

OK

Cancellation of payment notification (API call: IPCPurchaseCancel)

Purpose

This method is used by myPOS Checkout to notify the merchant that the customer has cancelled the payment. myPOS Checkout will redirect with this method when the customer chooses cancel payment. The call will be made on the previously supplied URL_Cancel.

Method properties

Property	Typical value	Type	Description
Amount	23.45	Double	Echo from IPCPurchase.
Currency	EUR	A(3)	Echo from IPCPurchase.
OrderID	201203319999999	string	Echo from IPCPurchase.



Signature	Byte[]	BASE64	SHA-256 HASH for all properties in the command. Signature is ALWAYS THE LAST PARAMETER IN THE POST , as it is not used to calculate the hash.
-----------	--------	--------	--

Rollback of previous notification (API call: IPCPurchaseRollback)

Purpose

This method is used by myPOS Checkout to notify that a reversal is passed for previous successful authorization. The merchant should mark the order as not paid (in case the merchant has received IPCPurchaseNotify method). This is used when myPOS Checkout does not receive an HTTP OK from the merchant as a response for IPCPurchaseNotify method. The call will be posted to URL_Notify.

Method properties

Property	Typical value	Type	Description
Amount	23.45	Double	Echo from IPCPurchase.
Currency	EUR	A(3)	Echo from IPCPurchase.
OrderID	201203319999999	string	Echo from IPCPurchase.
Signature	Byte[]	BASE64	SHA-256 HASH for all properties in the command. Signature is ALWAYS THE LAST PARAMETER IN THE POST , as it is not used to calculate the hash.

In-app Purchase with payment card (API call: IPCIAPurchase)

Purpose

This method initiates the beginning of the payment process for a customer (purchase). However the customer will remain within the pages of the external application. All needed data (incl. payment card details) will be collected and submitted to myPOS Checkout API by the external application. The method can be used both for a recurring transaction with already stored card or for an initial card payment. The myPOS Checkout API will return an xml with the result.

myPOS Checkout will check for:

- Valid myPOS Account number
- Valid Online Store ID corresponding with this myPOS Account number
- Valid status of the Online store (enabled)
- Valid currency and total amount
- Valid signature

Method properties

Property	Typical value	Type	Required	Description
OrderID	20120331999999	String	YES	Placeholder for the merchant. Used to put some data that will help the merchant to recognize for which order is the payment. Up to 255 characters.
Amount	23.45	Double	YES	The amount of the payment requested.
Currency	EUR	A(3)	YES	ISO 3-character currency code. The currency for the payment should be registered and approved.
CardType	1	N(1)	Conditional	<i>Conditional.</i> CardType is required when CardToken is not provided in the request. Please refer to Appendix IV .
PAN	Byte[]	BASE64	Conditional	<i>Conditional.</i> PAN is required when CardToken is not provided in the request.
CardholderName	John Smith	String (30)	Conditional	<i>Conditional.</i> CardholderName is required when CardToken is not provided in the request.
ExpDate	Byte[]	BASE64	Conditional	<i>Conditional.</i> ExpDate is required when CardToken is not provided in the request.
CVC	Byte[]	BASE64	Conditional	<i>Conditional.</i> CVC is required when CardToken is not provided in the request.
ECI	6	N(1)	Conditional	<i>Conditional.</i> ECI is required when CardToken is not provided in the request.
AVV	Byte[]	BASE64	Conditional	<i>Conditional.</i> AVV is required when CardToken is not provided in the request and the card is 3DS enabled.
XID	Byte[]	BASE64	Conditional	<i>Conditional.</i> AVV is required when CardToken is not provided in the request and the card is 3DS enabled.
CardToken	Byte[]	BASE64	Conditional	<i>Conditional.</i> The CardToken is required when the card is already stored and no additional data is sent for the card within the request.
AccountSettlement	11111111119	N(11)	NO	Account for payment settlement
Note		String	NO	Text associated with the purchase.

CartItems	2	Int	YES	The number of rows (items) in the logical record Cart. If there will be some additional fees/taxes for the cardholder, they need to be added as new items.
<u>Cart</u>	Logical Holder	Logical Record	YES	Array provided by the Merchant. The array describes the content of the shopping cart. The content will be displayed on the myPOS Checkout payment page.
OutputFormat	xml	String	NO	Output format of data. The property can be "xml" or "json". If it is not specified in the request, the default value is "xml".

Cart Logical Record

Cart logical record consists of standard POST parameters with the form name=value. For each consequent item an index is added that shows the logical record number for the item (ex. Article_1). Indexes are from 1 to <CartItems>.

Property	Typical value	Type	Description
Article	HP ProBook 6360b sticker	String	Name of an article in the shopping cart.
Quantity	2	Int	How many pieces of an article.
Price	2.34	Double	Price of a single unit.
Amount	4.68	Double	Quantity*Price for the article.
Currency	EUR	A(3)	Should be the same currency as in the purchase amount.

Response properties

Property	Typical value	Type	Description
Amount	23.45	Double	Echo from IPCIAPurchase
Currency	EUR	A(3)	Echo from IPCIAPurchase
OrderID	201203319999999	string	Echo from IPCIAPurchase
IPC_Trnref	12345678923	String	Used to uniquely identify a transaction in IPC. Used as a parameter for subsequent refund of reversal if needed.

Example of the xml response

```
<IPC_response>
  <IPCmethod>IPCIAPurchase</IPCmethod>
  <IPC_Trnref>1484134559</IPC_Trnref>
  <Amount>50</Amount>
  <Currency>USD</Currency>
  <Status>0</Status>
  <StatusMsg>Success</StatusMsg>
  <Signature>b4sEVS+HKsUac3iwzYjl8PKrrPC19kuz+cNjWfo/+LmyWM6JXK9BoTwNcXrsVo/jOQORdT2ui0ZqwnbyNVckY
q1QZdnMQvsz2Xzph3PSO8gNCm/OiqKb0uYDkYXbF/Halm9iw51hX7p92xg/9g0IApexbuO1bUEGzBlkrZRelH0=</Signature>
```

</IPC_response>

Get transaction log for previously executed payment (API call: IPCGetTxnLog)

Purpose

This method is used by the Merchant to get information about a previously executed payment via IPCPurchase or IPCIAPurchase. The myPOS Checkout API will return an xml with detailed information about a specific OrderID. This method is intended to be utilized by the Merchant in his website back-end. The Merchant could decide whether or not to use this method.

Method properties

Property	Typical value	Type	Required	Description
OrderID	201203319999999	String	YES	Placeholder for the merchant. Used to put some data that will help the merchant to refer payments to orders. Up to 255 characters.
OutputFormat	xml	String	NO	Output format of data. The property can be "xml" or "json". If it is not specified in the request, the default value is "xml".

Response properties

Property	Typical value	Type	Required	Description
OrderID	201203319999999	String	YES	Placeholder for the merchant. Used to put some data that will help the merchant to refer payments to orders. Up to 255 characters.
log				Returns detailed information about a specific OrderID.

Example of the xml response

```
<ipc_response>
  <IPCmethod>IPCGetTxnLog</IPCmethod>
  <OrderID>201203319999999</OrderID>
  <log>
    <item>
      <time>11.03.2013 09:42:56</time>
      <action>Received new POST request</action>
      <result>
        <IPCmethod>IPCPurchase</IPCmethod>
        <IPCVersion>1.1</IPCVersion>
        <IPLanguage>en</IPLanguage>
        <SID>000000000000010</SID>
        <WalletNumber>61938166610</WalletNumber>
        <Amount>23.45</Amount>
        <Currency>EUR</Currency>
        <OrderID>201203319999999</OrderID>
        <URL_OK>http://site.ext/paymentOK<URL_OK>
      </result>
    </item>
  </log>
</ipc_response>
```

```

<URL_Cancel>http://site.ext/paymentNOK</URL_Cancel>
<URL_Notify>https://site.ext/paymentNotify</URL_Notify>
<KeyIndex>1</KeyIndex>
<CustomerEmail>name@website.com</CustomerEmail>
<CustomerPhone>+23568956958</CustomerPhone>
<CustomerFirstNames>John Santamaria</CustomerFirstNames>
<CustomerFamilyName>Smith</CustomerFamilyName>
<CustomerCountry>DEU</CustomerCountry>
<CustomerCity>Hamburg</CustomerCity>
<CustomerZIPCode>20095</CustomerZIPCode>
<CustomerAddress>Kleine Bahnstr. 41</CustomerAddress>
<Note>Some note here</Note>
<CartItems>2</CartItems>
<Article_1>HP ProBook 6360b sticker</Article_1>
<Quantity_1>1</Quantity_1>
<Price_1>20.00</Price_1>
<Currency_1>EUR</Currency_1>
<Amount_1>20.00</Amount_1>
<Article_2>Delivery</Article_2>
<Quantity_2>1</Quantity_2>
<Price_2>3.45</Price_2>
<Currency_2>EUR</Currency_2>
<Amount_2>3.45</Amount_2>
<Signature>WwSv6Hk3cH9VAnIfGxOkIY/Ph0psiWGywNuZl8dAYPQZa8qFrLDh5Ck1RDAE
7h1oSH0CZw5fVW7jAlhSGcjaOnk+wYbGVbIMxP59aUEKEG8wW0N2CH6oft92W/APQ60
rZ6Qsdlge3uiAKchfvJEcy3buuWVb6C2D58isBvrMFik=</Signature>
</result>
</item>
<item>
  <time>11.03.2013 09:42:56</time>
  <action>Checking method</action>
  <result>OK</result>
</item>
<item>
  <time>11.03.2013 09:42:56</time>
  <action>Checking signature</action>
  <result>OK</result>
</item>
<item>
  <time>11.03.2013 09:42:56</time>
  <action>Check if SID match to wallet number</action>
  <result>OK</result>
</item>
<item>
  <time>11.03.2013 09:42:56</time>
  <action>Checking is purchase paid on Start page</action>
  <result>OK</result>
</item>
<item>
  <time>11.03.2013 09:42:56</time>
  <action>Checking is amount > 0</action>
  <result>OK</result>

```

```

</item>
</item>
<item>
  <time>11.03.2013 09:42:58</time>
  <action>Checking for request marked as paid with this order_id</action>
  <result>OK</result>
</item>
<item>
  <time>11.03.2013 09:43:02</time>
  <action>Checking is cart items > 0</action>
  <result>OK</result>
</item>
<item>
  <time>11.03.2013 09:43:06</time>
  <action>Checking the URLs (url_ok, url_cancel, url_notify)</action>
  <result>OK</result>
</item>
<item>
  <time>11.03.2013 09:43:26</time>
  <action>Checking cart</action>
  <result>OK</result>
</item>
<item>
  <time>11.03.2013 09:43:26</time>
  <action>Check post params</action>
  <result>OK</result>
</item>
<item>
  <time>11.03.2013 09:43:46</time>
  <action>Checking method</action>
  <result>OK</result>
</item>
<item>
  <time>11.03.2013 09:43:46</time>
  <action>Checking signature</action>
  <result>OK</result>
</item>
<item>
  <time>11.03.2013 09:43:46</time>
  <action>Check if SID match to wallet number</action>
  <result>OK</result>
</item>
<item>
  <time>11.03.2013 09:43:50</time>
  <action>Display page:</action>
  <result>Login or join</result>
</item>
<item>
  <time>11.03.2013 09:43:56</time>
  <action>Checking method</action>
  <result>OK</result>
</item>

```

```

<item>
  <time>11.03.2013 09:43:56</time>
  <action>Checking signature</action>
  <result>OK</result>
</item>
<item>
  <time>11.03.2013 09:43:56</time>
  <action>Check if SID match to wallet number</action>
  <result>OK</result>
</item>
<item>
  <time>11.03.2013 09:43:56</time>
  <action>Validate form data</action>
  <result>Required params: OK</result>
</item>
<item>
  <time>11.03.2013 09:43:56</time>
  <action> Validate form data </action>
  <result>PAN: OK</result>
</item>
<item>
  <time>11.03.2013 09:43:56</time>
  <action> Validate form data </action>
  <result>Expire year: OK</result>
</item>
<item>
  <time>11.03.2013 09:43:56</time>
  <action> Validate form data </action>
  <result>Expire date: OK</result>
</item>
<item>
  <time>11.03.2013 09:43:56</time>
  <action> Validate form data </action>
  <result>CVC: OK</result>
</item>
<item>
  <time>11.03.2013 09:43:56</time>
  <action> Validate form data </action>
  <result>Phone code match with country: OK</result>
</item>
<item>
  <time>11.03.2013 09:43:56</time>
  <action> Validate form data </action>
  <result>Validate phone: OK</result>
</item>
<item>
  <time>11.03.2013 09:43:56</time>
  <action> Validate form data </action>
  <result>Validate email: OK</result>
</item>
<time>11.03.2013 09:44:00</time>

```

```

    <action>Payment status</action>
    <result>0</result>
  </item>
  <item>
    <time>11.03.2013 09:44:00</time>
    <action>Sending POST request to Merchant with parameters:</action>
    <result>
      <IPCmethod>IPCPurchaseNotify</IPCmethod>
      <SID>000000000000010</SID>
      <Amount>23.45</Amount>
      <Currency>EUR</Currency>
      <OrderID>201203319999999</OrderID>
      <IPC_Trnref>370</IPC_Trnref>
      <RequestSTAN>000102</RequestSTAN>
      <RequestDateTime>2013-03-11 09:44:01</RequestDateTime>
      <Signature>mE0wMCIbyhtkwS3NNccO8iJK+RT2tsDnj9XolA6xA3bh+bPqglC4WBRD
      +IZFDRkIDFCh6tY4FvW8srbnmvlfwSOMmNqofe18mDnrDxAjy6Mj6yxCJReQsIOK
      IHTryAzAnEk7NbBBw/WcyptGQHFX9uBg3lsM/Pwn5E+IpxMiHxA=</Signature>
    </result>
  </item>
  <item>
    <time>11.03.2013 09:44:00</time>
    <action>Connecting with URL_NOTIFY</action>
    <result>Connection with http://site.ext/paymentNotify is OK</result>
  </item>
  <item>
    <time>11.03.2013 09:44:00</time>
    <action>Merchant response</action>
    <result>OK</result>
  </item>
  <item>
    <time>11.03.2013 09:44:00</time>
    <action>View "Thank You - Success" page</action>
    <result>OK</result>
  </item>
  <item>
    <time>11.03.2013 09:44:00</time>
    <action>Order done with status</action>
    <result>1</result>
  </item>
  <item>
    <time>11.03.2013 09:44:02</time>
    <action>Display page:</action>
    <result>Thank you page (Success)</result>
  </item>
</log>
<Status>0</Status>
<StatusMsg>Success</StatusMsg>
</ipc_response>

```

Detailed information about possible responses in the log is shown in the following table:

Response	Description
----------	-------------



IPCmethod	Name of the method requested for execution from myPOS Checkout.
Checking is purchase paid on Start page	
Checking Merchant for 3DS for card scheme	
Merchant is not 3DS	
Merchant is 3DS	
Client is redirected to "Response 3DS page".	
Display page	
Checking is amount > 0	Checking for a valid amount.
Checking valid currency and if the post currency is the currency of MID	
Checking for request marked as paid with this order_id	Checking for an existing already paid request with the same order_id.
Checking is cart_items > 0	
Checking the URLs (url_ok, url_cancel, url_notify)	
Checking cart	
Received new POST request	
Checking version	Checking for version number of myPOS Checkout.
Checking method	Checking for valid method.
Checking signature	Checking if parameter 'Signature' is correct.
Payment status	Returns information about the status of the payment.
View "Thank You - Success" page	
View "Thank You - Error" page	
Adding request in TODO table (cron)	Used in case where the party's response is different from 'OK' or reversal is not passed.
Checking reversal	
Checking clearing	
Validate form data	Checking for required fields, PAN, Expire date, CVC.
Sending POST request to Merchant with parameters	Returns information about the request.
Connecting with URL_NOTIFY	
Merchant response	
Start checking card for 3DS	
Starting payment (Without 3DS)	
Redirecting to ACS	Redirecting to Access control server in order to check for 3DSecure.
Error while checking card for 3DS	
Redirecting to Home Page	
Redirection to Merchant Cancel URL	
Redirection to Merchant OK URL	
Starting payment (With 3DS)	
Card successfully validated	
Card check for 3DS	
Invalid card issuer	
Check referrer URL	

Check post params	
Check 3DS response	
User is blocked by issuer	
Trying to send reversal notify (cron)	
Update cron table (cron)	
Order done with status	Returns the order's status (0 – pending, 1 – paid, 2 – reversed, 3 – cancelled).
Check if SID match to Wallet number	
Display output data	
OTHER	Other additional data.

Make a refund for previously executed payment (API call: IPCRefund)

Purpose

This method is used by the Merchant to initiate a refund of a previously executed payment. The myPOS Checkout API will return an xml with the result. This method is intended to be utilized by the Merchant in his website back-end. The Merchant could decide whether or not to use this method.

Method properties

Property	Typical value	Type	Required	Description
IPC_Trnref	12345678923	String	YES	Used to uniquely identify a transaction in IPC. Used as a parameter for subsequent refund or reversal if needed.
OrderID	201203319999999	String	NO	Placeholder for the merchant. Used to put some data that will help the merchant to recognize for which order is the payment. Up to 255 characters.
Amount	23.45	Double	YES	The amount of the payment requested.
Currency	EUR	A(3)	YES	ISO 3-character currency code. The currency for the payment should be registered and approved.
OutputFormat	xml	String	NO	Output format of data. The property can be "xml" or "json". If it is not specified in the request, the default value is "xml".

Response properties

Property	Typical value	Type	Required	Description
IPC_Trnref	12345678923	String	YES	Used to uniquely identify a transaction in IPC. Used as a parameter for subsequent refund or reversal if needed.
Amount	23.45	Double	YES	The amount of the payment requested.
Currency	EUR	A(3)	YES	ISO 3-character currency code. The currency for the payment should be registered and approved.



Example of the xml response

```
<IPC_response>
  <IPCmethod>IPCRefund</IPCmethod>
  <IPC_Trnref>12345678923</IPC_Trnref>
  <Amount>23.45</Amount>
  <Currency>EUR</Currency>
  <Status>0</Status>
  <StatusMsg>Success</StatusMsg>
</IPC_response>
```

Make a reversal for previously executed payment (API call: IPCReversal)

Purpose

This method is used by Merchant to initiate a reversal (void) of previously executed payment. The myPOS Checkout API will return an xml with the result. This method is intended to be utilized by the Merchant in his website back-end. The Merchant could decide whether or not to use this method.

Method properties

Property	Typical value	Type	Required	Description
IPC_Trnref	12345678923	String	YES	Used to uniquely identify a transaction in myPOS Checkout API. Used as a parameter for subsequent refund or reversal if needed.
OutputFormat	xml	String	NO	Output format of data. The property can be "xml" or "json". If it is not specified in the request, the default value is "xml".

Response properties

Property	Typical value	Type	Required	Description
IPC_Trnref	12345678923	String	YES	Used to uniquely identify a transaction in myPOS Checkout API. Used as a parameter for subsequent refund or reversal if needed.
Amount	23.45	Double	YES	The amount of the payment requested.
Currency	EUR	A(3)	YES	ISO 3-character currency code. The currency for the payment should be registered and approved.

Example of the xml response

```
<IPC_response>
  <IPCmethod>IPCReversal</IPCmethod>
  <IPC_Trnref>12345678923</IPC_Trnref>
  <Amount>23.45</Amount>
  <Status>0</Status>
  <StatusMsg>Success</StatusMsg>
</IPC_response>
```

Store card data for recurring payments (API call: IPCIAStoreCard)

Purpose

This method is used by myPOS Checkout API to allow cardholder to securely store card data which will be used for recurring payments afterwards. The cardholder will remain within the pages of the external application. All needed data (incl. payment card details) will be collected and submitted to myPOS Checkout API by the external app. The myPOS Checkout API will return an xml with the result.

Method properties

Property	Typical value	Type	Required	Description
CardType	1	N(1)	YES	
PAN	Byte[]	BASE64	YES	
CardholderName	John Smith	String (30)	YES	
ExpDate	Byte[]	BASE64	YES	
CVC	Byte[]	BASE64	YES	
ECI	6	N(1)	YES	
AVV	Byte[]	BASE64	Conditional	<i>Conditional.</i> AVV is required when the card is 3DS enabled.
XID	Byte[]	BASE64	Conditional	<i>Conditional.</i> AVV is required when the card is 3DS enabled.
CardVerification	2	N(2)	YES	Specify whether the inputted card data to be verified or not before storing. Please refer to Appendix V.
Amount	3.50	Double	Conditional	<i>Conditional.</i> Amount of the transaction. Used in the request if CardVerification = 2.
Currency	EUR	A(3)	Conditional	<i>Conditional.</i> ISO 3-character currency code. Used in the request if CardVerification = 2.
OutputFormat	xml	String	NO	Output format of data. The property can be "xml" or "json". If it is not specified in the request, the default value is "xml".

Response properties

Property	Typical value	Type	Description
CardToken	1041333312721BC752C1AB77 43D0821AA1C9CA09	String (50)	Uniquely generated token of the PAN of the card (returned as a response property of IPCIAPurchase).
PAN	4885	N(4)	Last four digits of the account number (PAN).
ExpDate	1703	N(4)	Card expire date. Format YYMM.
CardType	1	N(1)	Please refer to Appendix IV .

Example of the xml response

```
<IPC_response>
  <IPCmethod>IPCIASStoreCard</IPCmethod>
  <CardToken>70b2214c43814a99f66266c1f1f1e5ec4a8e2a01</CardToken>
  <PAN>4885</PAN>
  <ExpDate>1703</ExpDate>
  <CardType>1</CardType>
  <Status>0</Status>
  <StatusMsg>Success</StatusMsg>
</IPC_response>
```

Update data of stored card for recurring payments (API call: IPCIASStoredCardUpdate)

Purpose

This method is used by myPOS Checkout API to allow cardholder to securely update the expiry date, cardholder name and the Custom name of already stored card. The customer will remain within the pages of the external application. All needed data (incl. payment card details) will be collected and submitted to myPOS Checkout API by the external app. The myPOS Checkout API will return an xml with the result.

Method properties

Property	Typical value	Type	Required	Description
CardToken	1041333312721B C752C1AB7743D0 821AA1C9CA09	String (50)	YES	Uniquely generated token of the PAN of the card (returned as a response property of IPCIAPurchase).
CardholderName	John Smith (30)	String (30)	YES	
CardType	1	N(1)	YES	Please refer to Appendix IV .
ExpDate	Byte[]	BASE64	YES	
CVC	Byte[]	BASE64	YES	
ECI	6	N(1)	YES	
AVV	Byte[]	BASE64	Conditional	<i>Conditional.</i> AVV is required when the card is 3DS enabled.
XID	Byte[]	BASE64	Conditional	<i>Conditional.</i> AVV is required when the card is 3DS enabled.
CardVerification	2	N(2)	YES	Specify whether the card data to be verified or not before storing. Please refer to Appendix V .
Amount	23.45	Double	Conditional	<i>Conditional.</i> Amount of the transaction. Used in the request if CardVerification = 2.
Currency	EUR	A(3)	Conditional	<i>Conditional.</i> ISO 3-character currency code. Used in the request if CardVerification = 2.



OutputFormat	xml	String	NO	Output format of data. The property can be “xml” or “json”. If it is not specified in the request, the default value is “xml”.
--------------	-----	--------	----	--

Response properties

Property	Typical value	Type	Description
CardToken	1041333312721BC752C1AB7743D0821AA1C9CA09	String (50)	Uniquely generated token of the PAN of the card (returned as a response property of IPCIAPurchase).
PAN	4885	N(4)	Last four digits of the account number (PAN).
ExpDate	1703	N(4)	Card expire date. Format YYMM.
CardType	1	N(1)	Please refer to Appendix IV .

Example of the xml response

```
<IPC_response>
  <IPCmethod>IPCIASStoredCardUpdate</IPCmethod>
  <PAN>4885</PAN>
  <ExpDate>1703</ExpDate>
  <CardType>1</CardType>
  <Status>0</Status>
  <StatusMsg>Success</StatusMsg>
</IPC_response>
```

Request money from a client's myPOS account (API call: IPCRequestMoney)

Purpose

This method is used by the merchant to initiate a direct withdrawal transaction from a client's myPOS account. The mandate with the client must be registered within the myPOS system prior to initiation of the call IPCRequestMoney. Each client's mandate reference must be registered via the call IPCMandateManagement.

The merchant must have fully signed contract with myPOS for the service in order to be able to use this call.

The myPOS Checkout API will return an xml with the result. This method is intended to be utilized by the merchant in his website back-end.

Method properties

Property	Typical value	Type	Required	Description
MandateReference	126ca831-93d2-4dfc-ab1f-0cce1d0abe9e	String	YES	A unique identifier of the agreement (mandate) between the merchant and the client (debtor). Up to 127 characters.
CustomerWalletNumber	61938166612	AN(11)	YES	Identifier of the client's (debtor's) myPOS account

OrderID	201203319999999	String	YES	Placeholder for the merchant. Used to put some data that will help the merchant to recognize for which order is the payment. Up to 255 characters.
ReversalIndicator	1	N(1)	NO	Used only if there is a system error or no response was received. The Merchant must submit the same data with IPCRequestMoney method including this parameter with value = 1 (Reversal of the previously executed Request money transaction).
Amount	23.45	N(18,2)	YES	The amount of the payment requested.
Currency	EUR	A(3)	YES	ISO 3-character currency code.
Reason	Invoice num 123456	String	YES	The reason for the transfer.
OutputFormat	xml	String	NO	Output format of data. The property can be "xml" or "json". If it is not specified in the request, the default value is "xml".

Example of the xml response

```
<IPC_response>
  <IPCmethod>IPCRequestMoney</IPCmethod>
  <MandateReference>126ca831-93d2-4dfc-ab1f-0cce1d0abe9e </MandateReference>
  <CustomerWalletNumber>61938166612</CustomerWalletNumber>
  <IPC_Trnref>12345678923</IPC_Trnref>
  <ReversalIndicator></ReversalIndicator>
  <Amount>23.45</Amount>
  <Currency>EUR</Currency>
  <Status>0</Status>
  <StatusMsg>Success</StatusMsg>
</IPC_response>
```



In case of system error or missing response the merchant must call the method IPCRequestMoney with ReversalIndicator included. The reversal of the transaction can be processed if there are enough money at the merchant's account.

Manage registrations for Request money transactions (API call: IPCMandateManagement)

Purpose

This method is used by the merchant to register a mandate reference for direct withdrawal transactions (Request money) from a client's myPOS account or to cancel a previously registered mandate reference. The merchant can register more than one mandate reference for each client. All mandate references are registered to a specific merchant's myPOS account.

The myPOS Checkout API will return an xml with the result. This method is intended to be utilized by the merchant in his website back-end.

Method properties

Property	Typical value	Type	Required	Description
MandateReference	126ca831-93d2-4dfc-ab1f-0cce1d0abe9e	String	YES	A unique identifier of the agreement (mandate) between the Merchant and the client (debtor). Up to 127 characters.
CustomerWalletNumber	61938166612	AN(11)	YES	Identifier of the client's (debtor's) myPOS account
Action	1	N(1)	YES	1 – Registration of the MandateReference; 2 – Cancellation of a MandateReference
MandateText	This is what we agreed ...	String	NO	Text supplied from the merchant, so the client can easily identify the Mandate.
OutputFormat	xml	String	NO	Output format of data. The property can be "xml" or "json". If it is not specified in the request, the default value is "xml".

Example of the xml response

```
<IPC_response>
  <IPCmethod>IPCMandateManagement</IPCmethod>
  <CustomerWalletNumber>61938166612</CustomerWalletNumber>
  <MandateReference>126ca831-93d2-4dfc-ab1f-0cce1d0abe9e </MandateReference>
  <Action>1</Action>
  <Status>0</Status>
  <StatusMsg>Success</StatusMsg>
</IPC_response>
```

Get transaction status for previously executed payment (API call: IPCGetTxnStatus)

Purpose

This method is used by Merchant to get the current status of a previously executed payment. The myPOS Checkout API will return an xml with detailed information about a specific OrderID. This method is intended to be utilized by the Merchant in his website back-end. The Merchant could decide whether or not to use this method.

Method properties

Property	Typical value	Type	Required	Description
OrderID	201203319999999	String	YES	Placeholder for the merchant. Used to put some data that will help the merchant to refer payments to orders. Up to 255 characters.

OutputFormat	xml	String	NO	Output format of data. The property can be “xml” or “json”. If it is not specified in the request, the default value is “xml”.
--------------	-----	--------	----	--

Response properties



Please refer to Response properties of every command in order to see the information which will be returned by myPOS Checkout API.

Make a credit note to a myPOSWallet (API call: IPCSendMoney)

Purpose

This method is used by the Merchant to initiate a credit note to myPOSWallet. The IPC API will return an xml with the result. This method is intended to be utilized by the Merchant in his website back-end. The Merchant could decide whether or not to use the method.

Method properties

Property	Typical value	Type	Required	Description
CustomerWalletNumber	61938166610	AN(11)	YES	Identifier of myPOS Wallet number
Amount	23.45	N(18,2)	YES	The amount of the payment requested.
Currency	EUR	A(3)	YES	ISO 3-character currency code. The currency for the payment should be registered and approved.
TransactionReference	12345678923	String	YES	Used to uniquely identify a transaction in IPC. Used as a parameter for subsequent refund of reversal if needed.
Reason	Invoice num 123456	String	Yes	The reason for the transfer.
OutputFormat	xml	String	NO	Output format of data. The property can be “xml” or “json”. If it is not specified in the request, the default value is “xml”.

Example of the xml response

```
<IPC_response>
  <IPCmethod>IPCSendMoney</IPCmethod>
  <Amount>23.45</Amount>
  <Currency>EUR</Currency>
  <IPC_Trnref>12345678923</IPC_Trnref>
  <CustomerWalletNumber>61938166610</CustomerWalletNumber>
  <Status>0</Status>
  <StatusMsg>Success</StatusMsg>
</IPC_response>
```



Accepting online payments

myPOS Checkout API v.1.4



Accepting online payments

myPOS Checkout API v.1.4

Appendix I – Error messages

Code	Error	Description
0	Success	No errors
1	E_MISSING_REQ_PARAMS	Some of the required fields from the POST request are missing.
2	E_SIGNATURE_FAILED	The parameter 'Signature' is not correct.
3	E_IPC_ERROR	Invalid or missing response from myPOS Checkout servers. Please contact myPOS engineers.
4	E_INVALID_SID	The parameter 'SID' is not correct.
5	E_INVALID_PARAMS	One or more of the other parameters from the POST request are not correct.
6	E_INVALID_REFERER	The POST request is received from an URL which is not approved.
8	E_TRANSACTION_AUTH_FAIL	The authorization of the transaction has been failed.
9	E_WRONG_AMOUNT	The parameter 'Amount' is not correct.
10	E_UNSUPPORTED_CALL	
11	E_INACTIVE_MANDATE_REFERENCE	The error will be returned both if the registration of the mandate reference is still not authorized, rejected or temporarily blocked.
12	E_INVALID_MANDATE_REFERENCE	
13	E_NOT_SUFFICIENT_FUNDS	
14	E_TRANSACTION_NOT_PERMITTED	
15	E_EXCEEDED_LIMIT	
16	E_MANDATE_ALREADY_REGISTERED	
17	E_INACTIVE_ACCOUNTIDENTIFIER	The error will be returned when the client's payment instrument is still not activated.
18	E_INVALID_ACCOUNTIDENTIFIER	
19	E_EXCEEDED_ACCOUNT_LIMITS	The error will be returned when the transaction cannot be processed because of reached Wallet Account limits.
20	E_DUPLICATE_TRANSMISSION	
21	E_TRANSACTION_DECLINED	
99	E_UNDEFINED_ERROR	Other unspecified error.

Note:

If the system returns an error on IPCPurchase call, the payment page will not be displayed. The cardholder (client) will see an error page with the following general message:

[The online store name] has sent a shopping cart with errors in it.

We will contact the Merchant with a request to fix this problem. As this could be a temporary issue, you can go back to try checking out again.

<< Return to [Online store name]



Appendix II – Testing data

Field name	Test value
myPOS Checkout payment link	https://www.mypos.eu/vmp/checkout-test
SID	0000000000000010
myPOS Account Number	61938166610

Test private key

```
-----BEGIN RSA PRIVATE KEY-----
MIICXAIBAAKBgQCf0TdcTuph7X+Zwekt1XKEWZDczSGecfo6vQfqvraf5VPzcnJ
2Mc5J72HBm0u98EJHan+nle2WOZMVGIIta/2k1FRWwbt7iQ5dzDh5PEeZASg2UWe
hoR8L8MpNBqH6h7ZITwVTfRS4LSbVIEft7Pzhm5YJKfM+CdzDM+L9WVEGwIDAQAB
AoGAYfKxwUtebq8ulVrD3nnWhF+hk1k6KejdUqOdLYN29w8WjbCMKb9IaokmqWiQ
5iZGERYxh7G4BDP8AW/+M9HXM4oqm5SEkaxhbTlgks+E1s9dTpdFQvL76TvodqSy
l2E2BghVgLLgkdhRn9buaFzYta95JKfgyKGonNxsQA39PwECQQDKbG0Kp6KEkNgB
srCq3Cx2od5OfiPDG8g3RYZKx/O9dMy5CM160DwusVJpuywbpRhCWr3gkz0QgRMd
IRVwyxNbAkEAYh3sipmcgN7SD8xBG/MtBYpQWP1vxhSVYPfJzuPU3gS5MRJzQHBz
sVCLhTBY7hHSoqilqWYasi8JzBEwEuQQJBakw9qGcZjyMH8JU5TDSGllr3jybx
FFMPj8TgJs346AB8ozqLL/ThvWPpxHttJbH8QAdNuyWdg6dlfVAa95h7Y+MCQEZg
jRDI1Bz7eWGO2c0Fq9OTz3IVLWpnmGwFw+HyaxizxHv+FOj1GUVir9hylV7V0DU
Qjlajyv/oeDWhFQ9wQECQCydhJ6NaNQOCZh+6QTrH3TC5MeBA1Yeipoe7+BhsLNr
cFG8s9sTxRnltcZl1dXaBSemvpNvBizn0Kzi8G3ZAgc=
-----END RSA PRIVATE KEY-----
```

myPOS test public certificate

```
-----BEGIN CERTIFICATE-----
MIIBkDCB+qADAgECAGAwDQYJKoZIhvcNAQEFBQAwDzENMAAGA1UEChMEaVBheTAe
Fw0xMzAzMTMxMTI1MTFaFw0yMzAzMTExMTI1MTFaMA8xDTALBgNVBAoTBGIlQYXkw
gZ8wDQYJKoZIhvcNAQEBBQADgY0AMIGJAoGBAML+VTmiY4yChoOTMZTXAIG/mk+x
f/9mjwHxWzxtBJbNncNK0OLIOVXYKW2GgVklGHHQjvew1hTFkEGjnCJ7f5CDnbgx
evtyASDGst92a6xcAedEadP0nFXhUz+cYYIglcgfDcX3ZWeNEF5kscqy52kpD2O7
nFNCV+85vS4duJBNAgMBAAEwDQYJKoZIhvcNAQEFBQADgYEAfSfqJHH9Vp9Y4osJ
sLg1Um5LOoTgn6u4JepHMFoSiwYEOn/N3D3JlqgAzjdVJ+1rZV95VAf/+TKzWzvP
V8L01LJ8aRFkUaPGenVsGvBT2mtsbu34QUOIPgzCi3huidwk0yIMX7zo8uxu1cXv
/bg5jBGe5SjvJP8Tq257QcAGgkA=
-----END CERTIFICATE-----
```



Appendix III – Example for successful payment notification

Result POST data

```
Array (  
  [IPMethod] => IPCPurchaseNotify  
  [SID] => 0000000000000010  
  [Amount] => 50  
  [Currency] => EUR  
  [OrderID] => 1440146333  
  [IPC_Trnref] => 813705  
  [RequestSTAN] => 000006  
  [RequestDateTime] => 2015-08-21 10:39:37  
  [Signature] =>  
aPn96fYghN/hdLgP0oGbCglBACwJlG6TSIf7UJP9Qe9UMzBP9+B5r7enaLZNAWAHaws7iZH9INgQrHbnn/lalKFYzjWLHqCQm2xY  
PF4lAbIn5MC9UjfUXyMxZ42ENVx/Poeq0ZH1Zd34j2qbFJ8tViawUjey5BZRleJaEmQr3ww=  
);
```

Appendix IV – Card types

Code	Card type
1	MasterCard
2	Maestro
3	VISA
4	Visa Electron
5	VPAY
6	JCB
7	Bancontact

Appendix V – Card verification

Code	Description
1	The card data will be verified with a zero-amount debit transaction. Upon successful transaction the card data will be stored successfully.
2	The card data provided will be verified with a real debit transaction. Upon successful transaction the card data will be stored successfully.